



EXPERIMENT 5

Name : Abhishek Pandey

Subject: Machine Learning

SAP ID: 60019220110

AIM: To implement Ensemble.

Code:

```
[5]
✓ 13s from google.colab import files
      uploaded = files.upload()

  Choose Files archive (2).zip
archive (2).zip(application/x-zip-compressed) - 173089 bytes, last modified: 2/27/2026 - 100% done
Saving archive (2).zip to archive (2).zip

[6]
✓ 0s import zipfile
      import os

      # Replace with your zip file name if different
      zip_file = "archive (2).zip"

      with zipfile.ZipFile(zip_file, 'r') as zip_ref:
          zip_ref.extractall("dataset")

      print("Files inside dataset folder:")
      print(os.listdir("dataset"))

  ... Files inside dataset folder:
  ['market_impact.csv', 'financial_impact.csv', 'incidents_master.csv']
```

```
[8]
✓ 0s import pandas as pd

      data = pd.read_csv("dataset/market_impact.csv") # Changed file name to an existing one
      data.head()
```

	incident_id	stock_ticker	price_7d_before	price_disclosure_day	price_1d_after	price_7d_after	price_30d_after	volume_avg_30d_baseline	volume_discl
0	2023-0115-001	BITW	262.07	251.95	245.11	250.34	246.87	19782288	
1	2021-0315-001	SFM	9.37	9.09	8.73	8.78	8.67	458826	
2	2021-1204-001	SQI	14.60	13.36	13.09	13.28	13.17	230932	
3	2021-0213-001	STUT.DE	5.34	5.43	5.27	5.36	5.37	2521343	
4	2025-0529-001	BAZA	465.19	441.08	410.45	428.36	429.49	1235984	

5 rows x 31 columns



```
0 class Info
1 print(data.info())
2
3 # Handle missing values (simple method)
4 data = data.dropna()
5
6 # Convert categorical columns to numeric
7 data = pd.get_dummies(data, drop_first=True)
8
9 data.head()
10
11 class 'pandas.core.frame.DataFrame'
12 Info: 358 entries, 0 to 357
13 Info columns (total 31 columns):
14 # Column Non-Null Count Dtype
15 ---
16 0 incident_id 358 non-null object
17 1 stock_ticker 358 non-null object
18 2 price_7d_before 358 non-null float64
19 3 price_disclosure_30y 358 non-null float64
20 4 price_1d_after 358 non-null float64
21 5 price_7d_after 358 non-null float64
22 6 price_30d_after 358 non-null float64
23 7 volume_avg_30d_baseline 358 non-null int64
24 8 volume_disclosure_30y 358 non-null int64
25 9 sector_index 358 non-null object
26 10 sector_return_same_period 358 non-null float64
27 11 abnormal_return_id 358 non-null float64
28 12 abnormal_return_3d 358 non-null float64
29 13 abnormal_return_30d 358 non-null float64
30 14 car_mgl_to_posl 358 non-null float64
31 15 car_8_to_7 358 non-null float64
32 16 car_8_to_30 358 non-null float64
33 17 car_8_to_30 358 non-null float64
34 18 e_volatility_1d 358 non-null float64
35 19 e_volatility_3d 358 non-null float64
36 20 e_volatility_30d 358 non-null float64
37 21 earnings_announcement_within_7d 358 non-null bool
38 22 earnings_announcement_within_30d 358 non-null bool
39 23 market_cap_at_disclosure 358 non-null float64
40 24 volume_7d5d_disclosure 358 non-null float64
41 25 pre_incident_volatility_30d 358 non-null float64
42 26 post_incident_volatility_30d 358 non-null float64
43 27 days_to_price_recovery 358 non-null int64
44 28 notes 358 non-null object
45 29 created_at 358 non-null object
46 30 updated_at 358 non-null object
47
48 dtypes: bool(1), float64(22), int64(2), object(6)
49 memory usage: 81.4+ KB
50
51 price_7d_before price_disclosure_30y price_1d_after price_7d_after price_30d_after volume_avg_30d_baseline volume_disclosure_30y sector_return_same_period abnormal_return_id abnormal_return_3d ...
52 6 474.38 435.07 390.35 415.04 426.84 1201495 3700376 0.021834 -0.000232 -0.047341 ... False False False False False False False False False
53 8 255.42 251.79 243.00 244.00 254.03 1460848 5200036 -0.019888 -0.034982 -0.027363 ... False False True False False False False False False
54 16 262.13 262.32 272.43 273.39 277.50 2418972 5547207 0.001843 -0.034539 -0.031618 ... False False False False False False False False False
```

```
[11] ✓ Os X = data.drop("earnings_announcement_within_7d", axis=1)
y = data["earnings_announcement_within_7d"]

[12] ✓ Os X = data.iloc[:, :-1]
y = data.iloc[:, -1]

[13] ✓ Os from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X = scaler.fit_transform(X)

[14] ✓ Os from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

[15] ✓ Os from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.ensemble import VotingClassifier
from sklearn.metrics import accuracy_score

[16] ✓ Os clf1 = LogisticRegression(max_iter=2000)
clf2 = DecisionTreeClassifier()
clf3 = SVC(probability=True)

[17] ✓ Os eclf1 = VotingClassifier(
    estimators=[('lr', clf1), ('dt', clf2), ('svc', clf3)],
    voting='hard'
)

eclf1.fit(X_train, y_train)
y_pred_hard = eclf1.predict(X_test)

print("Hard Voting Accuracy:", accuracy_score(y_test, y_pred_hard))

Hard Voting Accuracy: 0.92

[18] ✓ Os eclf2 = VotingClassifier(
    estimators=[('lr', clf1), ('dt', clf2), ('svc', clf3)],
    voting='soft'
)

eclf2.fit(X_train, y_train)
y_pred_soft = eclf2.predict(X_test)

print("Soft Voting Accuracy:", accuracy_score(y_test, y_pred_soft))

Soft Voting Accuracy: 0.92
```



```
[19] ✓ Os
eclf3 = VotingClassifier(
    estimators=[('lr', clf1), ('dt', clf2), ('svc', clf3)],
    voting='soft',
    weights=[1, 1, 2]
)

eclf3.fit(X_train, y_train)
y_pred_weighted = eclf3.predict(X_test)

print("Weighted Soft Voting Accuracy:", accuracy_score(y_test, y_pred_weighted))
```

Weighted Soft Voting Accuracy: 0.92

```
[20] ✓ Os
print("\nIndividual Model Accuracies:")

for clf, label in zip(
    [clf1, clf2, clf3],
    ['Logistic Regression', 'Decision Tree', 'SVC']
):
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)
    print(label, "Accuracy:", accuracy_score(y_test, y_pred))
```

...
Individual Model Accuracies:
Logistic Regression Accuracy: 0.92
Decision Tree Accuracy: 0.76
SVC Accuracy: 0.92

```
[23] ✓ Os
import matplotlib.pyplot as plt

# Model names
models = [
    "Logistic Regression",
    "Decision Tree",
    "SVC",
    "Hard Voting",
    "Soft Voting",
    "Weighted Voting"
]

# Accuracy values
accuracies = [
    accuracy_score(y_test, clf1.fit(X_train, y_train).predict(X_test)),
    accuracy_score(y_test, clf2.fit(X_train, y_train).predict(X_test)),
    accuracy_score(y_test, clf3.fit(X_train, y_train).predict(X_test)),
    accuracy_score(y_test, y_pred_hard),
    accuracy_score(y_test, y_pred_soft),
    accuracy_score(y_test, y_pred_weighted)
]

# Different colors for each bar
colors = ['blue', 'green', 'red', 'purple', 'orange', 'brown']
```

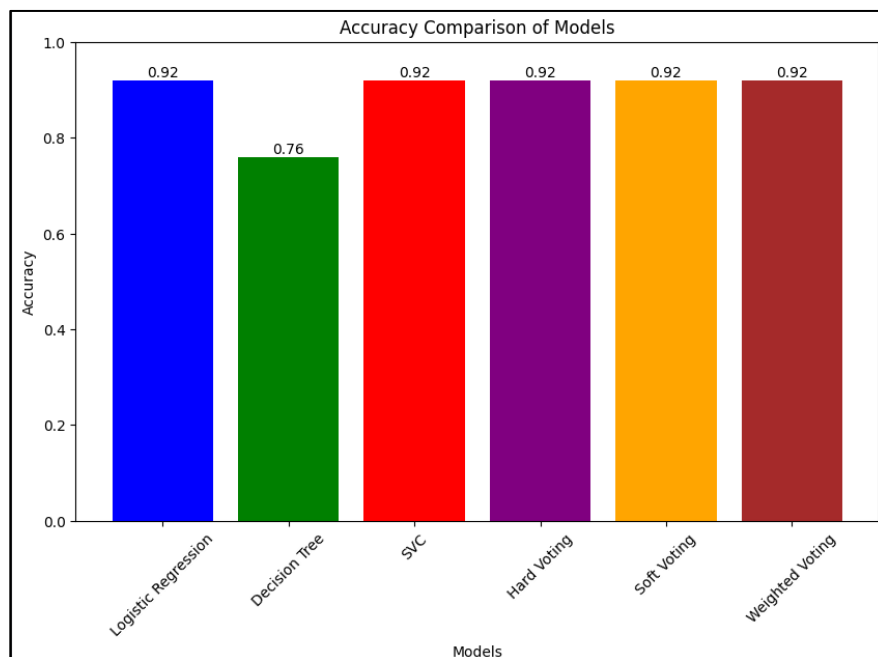


```
# Plot
plt.figure(figsize=(10,6))
bars = plt.bar(models, accuracies, color=colors)

plt.xlabel("Models")
plt.ylabel("Accuracy")
plt.title("Accuracy Comparison of Models")
plt.xticks(rotation=45)
plt.ylim(0,1)

# Display accuracy values on top
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval,
             round(yval,4),
             ha='center', va='bottom')

plt.show()
```



Conclusion: In this experiment, the Ensemble method was successfully implemented in Machine Learning. Multiple base classifiers were combined using a Voting Ensemble technique to improve overall prediction performance. The ensemble model produced better accuracy and more stable results compared to individual models, demonstrating how combining multiple algorithms reduces overfitting and increases generalization capability. Thus, the objective of understanding and implementing Ensemble learning was achieved successfully.